



SPIDER

PROGRAMMING LANGUAGE

Spider

PROGRAMMING LANGUAGE

</>

A Modern, Fast, and
Human-Friendly Language
for the Future



SIMPLE BY DESIGN

Clean syntax.
Easy to learn.



BLAZING FAST

Built for
performance.



MEMORY SAFE

Security without
compromise.



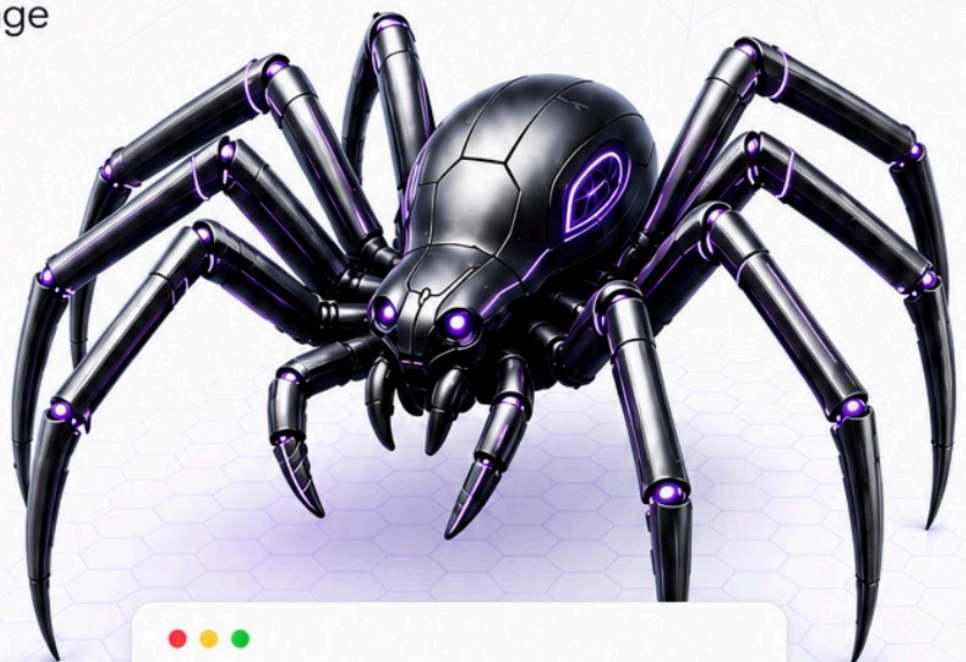
BUILT-IN TOOLS

Everything you need,
right out of the box.



EXTENSIBLE & FUTURE READY

Designed to grow
with you.



```
1 let greeting = "Welcome to Spider!"
2 fn main() {
3     say(greeting)
4     let x = 42
5     if x > 0 {
6         say("The future is Spider.")
7     }
8 }
9 >_
```

AUTHOR

KV Avinash Sarma

Founder and CEO

of Avixr Technologies Pvt. Ltd.



avixr.in



OPEN SOURCE
Community Driven



LEARN ONCE
Use Everywhere



BUILT FOR DEVELOPERS
Loved by Everyone



THE NEXT GENERATION
Programming Language

Spider

PROGRAMMING LANGUAGE

A modern, fast, and human-friendly language for the future.

by **KV Avinash Sarma**

Founder & CEO, Avixr Technologies Pvt. Ltd.

EARLY EDITION · V0.1.0

The Spider Programming Language

Early Edition — documenting Spider v0.1.0. This is a living book: Spider is under active development, and later editions will expand these chapters as the language grows toward 1.0.

Copyright © 2026 KV Avinash Sarma, Avixr Technologies Pvt. Ltd. All rights reserved.

Website & playground: spider-lang.vercel.app

Source code: github.com/kvAvinashsarma-dev/Spider-Programming-Language

Author: avixr.in

Written with care for everyone learning to program — from a curious beginner to a seasoned engineer.

Table of Contents

Welcome	5
Project Status	6
Part 1: Introduction	9
Vision and Philosophy	9
Spider Compared	11
Part 2: Installing Spider	13
Installing and Building	13
Part 3: Tools	15
Spider CLI	15
web Package Manager	17
LSP and Learn Mode	19
Part 4: Language Basics	21
Lexical Structure	21
Values, Variables, and Types	22
Expressions and Operators	24
Control Flow	25
Functions	27
Records, Choices, and Patterns	28
Collections and Outcomes	30
Modules and Packages	31
Testing	32
Concurrency Surface	33
Part 5: Standard Library	35
Standard Library Reference	35
Diagnostics Reference	38
Language Reference	40
CLI Reference	42
Bytecode Reference	43
Part 6: Internals	46
Compiler Architecture	46

Silk Virtual Machine	48
Formatter	49
Package Loader and Registry	50
Part 7: Learning by Building	52
Tutorial	52
Cookbook	54
Projects	56
Exercises and Answers	57
Part 8: Contributor Guide	62
Contributing	62
Adding Language Features	63
Appendix	65
Grammar Sketch	65
Keywords and Precedence	66
Glossary	67
FAQ	68
Troubleshooting	69
Validation Report	70

Welcome

Spider is a programming language created by Kv Avinash Sarma with a simple surface and an industrial core. Its repository currently contains the parser, formatter, semantic checker, Silk bytecode VM, command-line tool, local package manager, LSP, browser playground facade, corpus examples, milestone notes, and website.

This book is official documentation, not a marketing README. It documents what the implementation in this repository can parse, check, format, compile, and run today. When the roadmap and implementation disagree, this book follows the implementation and names the discrepancy.

A First Program

```
say "Hello, world!"

let name = ask "What is your name?"
say "Welcome, {name}!"
```

`say` prints a value. `ask` prints a prompt and returns text typed by the user. String interpolation uses `{...}` holes parsed as Spider expressions, with one current limitation: text literals inside interpolation holes are not supported because interpolation is still scanned inside string tokens.

The Shape of Spider Today

Spider files use the `.sp` extension. Blocks are indentation based. The formatter has no options. The parser is lossless and error tolerant. The checker performs name resolution, type checking, exhaustiveness checks, and capability checks. The compiler lowers checked programs to Silk bytecode. The VM executes with value semantics implemented by copy-on-write containers.

What This Book Includes

This book includes a tutorial, language reference, standard-library reference, diagnostics guide, CLI reference, package-manager guide, compiler internals guide, VM bytecode reference, contributor guide, projects, cookbook, exercises, answers, and a validation report.

Exercise

1. Write a file named `hello.sp` that asks for a name and prints a greeting.
2. Run it with `spider run hello.sp`.
3. Format it with `spider fmt hello.sp`.

Answer:

```
let name = ask "Name?"
say "Hello, {name}!"
```

Project Status

The repository is versioned as `0.1.0` and contains milestones M1 through M5 shipped, with M6 partially shipped.

Implemented

- Lossless lexer, parser, CST, typed AST views, and formatter.
- Semantic checker with name resolution, type inference, choices, records, generics, pattern matching, diagnostics, capabilities, and project modules.
- Silk register bytecode compiler and VM.
- `spider run`, `spider test`, `spider new`, `spider repl`, `spider fmt`, `spider check`, `spider tree`, `spider tokens`, `spider explain`, `spider learn`, and `spider lsp`.
- Local `web` package manager with `install`, `publish`, `audit`, and `remove`.
- Safe Mode capability model for `fs`, `net`, `env`, `exec`, and `ai`, with `files` implemented behind `fs`.
- LSP diagnostics, hover, and completion.

Not Implemented Yet

- `spider build` native compilation.
- `spider doc`.
- `spider package`.
- Public network registry, signing, namespaces, and transitive package dependency resolution.
- Full concurrency semantics. `do together` and `spawn` parse; runtime behavior is sequential until M7.
- AOT backend and WASM target as production outputs.
- Lexer-level interpolation tokens. Current interpolation is scanner based.
- Complete website documentation generated from this book.

Source of Truth

The code paths used for this book are:

- `crates/spider-syntax`
- `crates/spider-hir`
- `crates/spider-silk`
- `crates/spider-fmt`
- `crates/spider-web`
- `crates/spider-lsp`
- `crates/spider-playground`
- `crates/spider-cli`
- `corpus`

Project Status

- docs

PART 1

Introduction

Vision and Philosophy

Spider's thesis is that code can read plainly without becoming vague. It uses small, pronounceable keywords such as `say`, `ask`, `let`, `var`, `fn`, `match`, `repeat`, and `try`, but it is not natural-language programming. The grammar is deterministic and implemented by a recursive-descent parser.

Motivation

Spider is designed for learners, educators, production engineers, and tools. The compiler tries to teach. Diagnostics include stable codes and explanation entries. The formatter removes style debate. Safe Mode makes ambient power visible by requiring explicit capabilities.

Principles

- One obvious spelling for common ideas.
- Readable syntax with strict semantics.
- Errors as a product surface.
- Safe by default, capable by declaration.
- Deterministic builds and deterministic runtime behavior where possible.
- Tooling is part of the language, not an afterthought.

History

The repository records its early history through milestone notes:

- M1 Hatchling: syntax, parser, formatter.
- M2 Spinneret: HIR-style checking, inference, diagnostics.
- M3 Silk: bytecode compiler, VM, run, repl, new.
- M4 Web-spinning: stdlib registry, capabilities, tests.
- M5 First Thread: modules and local package manager.
- M6 Loom: partially shipped LSP and Learn Mode.

Behind the Scenes

The parser is lossless: every token, including comments and whitespace, remains in the tree. That is why the formatter, LSP, and future refactoring tools can operate without destroying user context.

Common Mistakes

- Treating Spider as English. It is not; it only borrows readable words.
- Assuming roadmap features exist. The compiler decides what is real.
- Expecting invisible package permissions. Capabilities are explicit.

Best Practices

- Run `spider check` before `spider run` when learning.
- Prefer `let` until mutation is required.
- Keep public functions annotated.
- Use `Outcome` for expected failure and let panics represent bugs.

Exercise

Explain why `say "hi"` is valid Spider but “print hi” is not.

Answer: Spider has a fixed grammar. `say` is the implemented output keyword; free English commands are not parsed.

Spider Compared

This chapter compares Spider’s implemented design with familiar languages.

Python

Spider shares indentation-based blocks and beginner-friendly readability with Python. Unlike Python, Spider is statically checked before execution and has explicit `Outcome` and `Maybe` values instead of exceptions and null.

Rust

Spider shares safety ambitions with Rust but does not expose borrow checking in the current user surface. Value semantics are implemented in the VM with copy-on-write containers. Spider’s capability model is also part of the language and package toolchain.

Go

Spider shares simple tooling and a preference for readable code with Go. Go’s current implementation has goroutines; Spider’s concurrency syntax exists but executes sequentially until M7.

Java

Spider avoids mandatory classes for simple programs. It has records and choices today, not class inheritance.

C#

C# has a broad industrial runtime and many language constructs. Spider is much smaller today. Its implemented surface favors few keywords, no nullable values, and deterministic package capability checks.

JavaScript

Spider is not dynamically typed JavaScript. It checks names, types, module visibility, match exhaustiveness, and capabilities before running.

Summary

Spider’s current identity is closest to “Python-like readability with a static checker, Rust-like honesty about safety, and a small integrated toolchain.”

PART 2

Installing Spider

Installing and Building

Spider is currently built from source with Cargo.

Requirements

- Rust toolchain with Cargo.
- A terminal.
- This repository.

Build

```
cargo build --workspace
```

The `spider` executable is produced under `target/debug/spider.exe` on Windows. The `web` package-manager executable is produced under `target/debug/web.exe`.

Run Tests

```
cargo test --workspace
```

PATH

During development, either run the executable by path:

```
.\target\debug\spider.exe --version
```

or add `target/debug` to your `PATH`.

Updating

Pull the repository, then rebuild:

```
git pull
cargo build --workspace
```

Troubleshooting

If Cargo cannot find Rust, install the Rust toolchain. If `spider` is not found, check whether `target/debug` is on `PATH` or run the executable by full path.

Exercise

Build the workspace and run `spider explain E0301`.

Answer: `E0301` is the divide-by-zero runtime panic code.

PART 3

Tools

Spider CLI

The `spider` command is implemented in `crates/spider-cli/src/main.rs`.

Commands Implemented Today

Command	Purpose
<code>`spider run <file.sp></code>	project-dir> `
<code>spider test [path]</code>	Run every <code>test "..."</code> block under a path.
<code>spider new <name></code>	Create a project with <code>web.toml</code> , <code>src/main.sp</code> , and <code>README</code> .
<code>spider repl [--allow cap]</code>	Interactive Spider session.
<code>spider fmt <paths...></code>	Format <code>.sp</code> files in place.
<code>spider fmt --check <paths...></code>	Report files that would be reformatted.
<code>spider check <file.sp></code>	Parse, resolve, type-check, and explain problems.
<code>spider tree <file.sp></code>	Print the syntax tree for debugging.
<code>spider tokens <file.sp></code>	Print the token stream for debugging.
<code>spider explain <CODE></code>	Print an explanation for a diagnostic code.
<code>spider learn <file.sp></code>	List concepts used by a file.
<code>spider lsp</code>	Run the language server over stdio.
<code>spider --version</code>	Show the toolchain version.

Safe Mode

Programs start with zero capabilities. Use `web.toml` or `--allow` to grant a capability for a run:

```
spider run --allow fs script.sp
spider repl --allow fs
```

Known capability names are `fs`, `net`, `env`, `exec`, and `ai`. Only `fs` has implemented standard-library operations today, through the `files` module.

Roadmap Commands Not Implemented

The public roadmap mentions `spider build`, `spider doc`, and package-oriented commands. In this repository:

- `spider build` is explicitly reported as coming later.
- `spider doc` is not implemented.
- Package operations live in the separate `web` executable.

Common Mistakes

- Running `spider check src` currently expects a file path, not a whole source directory.
- Running file I/O without `--allow fs` or a manifest grant produces E0244 at check time or E0310 at runtime.

Exercise

Create a project and run it.

Answer:

```
spider new hello-spider  
cd hello-spider  
spider run .
```

web Package Manager

`web` is Spider's package manager. The current implementation is an M5 local registry, not a public network registry.

Commands

Command	Purpose
<code>web install <name></code>	Install latest local-registry version into <code>web_modules</code> .
<code>web publish</code>	Publish the current project to the local registry.
<code>web audit</code>	Check installed packages, capabilities, and fingerprints.
<code>web remove <name></code>	Remove an installed package and lockfile entry.
<code>web --version</code>	Show the package manager version.

Manifest

`web.toml` records project metadata, capabilities, and dependencies:

```
[project]
name = "demo"
version = "0.1.0"
spider = "0.1"

[capabilities]
allow = []

[dependencies]
```

Capability Diffing

Installation refuses a package whose capabilities exceed the project allow-list. This is the security center of M5. A dependency that wants `fs` cannot be installed into a project whose manifest allows no capabilities.

Lockfile

`web.lock` records exact package versions and deterministic FNV-64 content fingerprints. The fingerprint is an integrity check for local tampering, not a cryptographic signature.

Limitations

- Registry is local: `~/.spider/registry` or `SPIDER_REGISTRY`.
- Published versions are immutable.
- Transitive package dependencies are not resolved.

- Public registry, signing, namespace ownership, semver enforcement, and vulnerability feeds are future work.

Exercise

Why are install scripts omitted?

Answer: Spider's package model treats packages as data. Arbitrary install-time code would undermine deterministic builds and capability security.

LSP and Learn Mode

M6 partially shipped `spider lsp` and `spider learn`.

Language Server

The LSP server communicates over stdio with Content-Length framing. It supports:

- initialize/shutdown lifecycle;
- diagnostics on open and change;
- keyword and stdlib hover text;
- keyword and stdlib completion.

It intentionally reuses the same parsing and checking pipeline as `spider check`, so editor diagnostics match terminal diagnostics.

Learn Mode

`spider learn file.sp` scans a file and lists concepts it uses. The concept database also powers LSP keyword hovers.

Example

```
spider learn corpus\run\loops.sp
```

Limitations

- Cross-file LSP project checking is not complete.
- Kids Mode is not implemented as a separate mode.
- In-string completions wait on lexer-level interpolation tokens.

PART 4

Language Basics

Lexical Structure

Spider source is UTF-8 text in `.sp` files. The current lexer accepts ASCII identifiers beginning with a letter or `_`, followed by letters, digits, or `_`.

Comments

```
# A normal comment.
## A documentation comment.
```

Strings

```
let greeting = "Hello"
let message = "Hello, {name}!"
```

Escapes supported by the interpolation scanner include `\n`, `\t`, `\"`, `\\`, `\{`, and `\}`.

Numbers

```
let whole = 1_000_000
let decimal = 3.14
let tiny = 2.5e-3
let negative = -42
```

`Int` is 64-bit checked. `Float` follows IEEE behavior and does not panic for divide-by-zero.

Indentation

Blocks are indentation based. Tabs in indentation produce E0001. Indentation must line up with an earlier indentation level.

```
if true
  say "indented"
```

Soft Keywords

`record`, `choice`, `shape`, `test`, `times`, `together`, and `where` are contextual. They can be names when a name is expected:

```
let times = 3
repeat times times
  say "again"
```

Hard structural words such as `let`, `var`, `fn`, `if`, `else`, `for`, `while`, `match`, `try`, `use`, `return`, `say`, `ask`, `spawn`, `do`, `and`, `or`, `not`, `true`, `false`, `public`, `to`, `of`, `is`, and `in` are reserved.

Values, Variables, and Types

Spider has immutable bindings, mutable variables, primitive values, collections, records, choices, and typed failure values.

Bindings

```
let pi = 3.14159
var score = 0
score += 5
```

Use `let` by default. Use `var` only when the name must be reassigned or mutated.

Type Annotations

```
let title: Text = "Spider"
let scores: List of Int = [90, 85, 77]
let ages: Map of Text to Int = {"Ada": 12, "Lin": 11}
```

The checker infers local `let` types. Public function boundaries must be annotated.

Primitive Types

- `Int`
- `Float`
- `Bool`
- `Text`
- `Nothing`

Generic Types

```
List of Int
Map of Text to Int
Maybe of Text
Outcome of Int
```

Value Semantics

Assignment behaves like a copy. The VM implements this with copy-on-write.

```
var a = [1, 2]
var b = a
b.push(3)
say a
say b
```


Output:

```
[1, 2]  
[1, 2, 3]
```

Common Mistakes

- Assigning to a `let` binding.
- Mixing `Int` and `Float` without conversion.
- Leaving public function parameters unannotated.

Expressions and Operators

Arithmetic

```
say 1 + 2 * 3
say (1 + 2) * 3
say 10 / 2
say 10 % 3
```

Integer divide-by-zero panics with E0301. Integer overflow panics with E0302.

Comparison and Logic

```
if age > 3 and age < 13 or not false
  say "boolean logic reads like a sentence"
```

Comparison operators are `==`, `!=`, `<`, `<=`, `>`, and `>=`.

Calls

```
say area(3.0, 4.0)
say "hello".upper()
say [3, 1, 2].sort()
```

User functions and methods use parentheses. The built-in `say` and `ask` forms are prefix forms.

Indexing and Assignment

```
var scores = [90, 85]
scores[1] = 100

var ages = {"Ada": 12}
ages["Sam"] = 13
```

List indexes outside range panic with E0303. Missing map keys panic with E0304.

Ranges

```
for i in 1 to 10
  say i
```

`to` forms an inclusive range in current loop behavior.

Control Flow

if and else

```
if age >= 13
  say "Teenager or older"
else if age >= 3
  say "Kid"
else
  say "Toddler"
```

Conditions must be `Bool` .

while

```
var lives = 3
while lives > 0
  say lives
  lives -= 1
```

for

```
for i in 1 to 3
  say i

for item in [1, 2, 3]
  say item * 2
```

repeat

```
repeat 3 times
  say "Hip hip hooray!"
```

The expression before `times` must be an `Int` .

match

```
match parse_age("")
  Ok(value) -> say value
  Fail(problem) -> say "could not parse: {problem}"
```

The checker performs exhaustiveness checks for choices, `Maybe` , `Outcome` , and `Bool` .

return

```
fn double(n: Int) -> Int  
  return n * 2
```

Functions with non-`Nothing` return types may also end with a trailing expression.

Not Implemented

There is no implemented `break` or `continue` command in the current grammar.

Functions

Declaration

```
fn greet(name: Text)
  say "Hello, {name}!"
```

Return Types

```
fn area(width: Float, height: Float) -> Float
  return width * height
```

Public Functions

```
public fn total(prices: List of Float) -> Float
  var sum = 0.0
  for price in prices
    sum += price
  return sum
```

Public functions cross module boundaries. Public parameters must have annotations.

Generics

```
fn largest(items: List of T) -> Maybe of T where T is Comparable
  return items.first()
```

Implemented built-in constraints are `Comparable`, `Equatable`, and `Printable`.

Recursion

```
fn fib(n: Int) -> Int
  if n < 2
    return n
  return fib(n - 1) + fib(n - 2)
```

The VM uses explicit frames. Recursion is limited to 1000 calls and reports E0307 when exceeded.

Closures

Closures and anonymous functions are not implemented in the current grammar.

Records, Choices, and Patterns

Records

```
record Point
  x: Float
  y: Float

let p = Point(3.0, 4.0)
say p.x
```

Records are constructed with call syntax. Fields are accessed with dot syntax.

Choices

```
choice Shape
  Circle(radius: Float)
  Rect(width: Float, height: Float)
  Dot
```

Choices are tagged alternatives. Constructors also use call syntax when they carry fields.

Pattern Matching

```
fn area(shape: Shape) -> Float
  match shape
    Circle(r) -> 3.14159 * r * r
    Rect(w, h) -> w * h
    Dot -> 0.0
```

Patterns can match variant tags and bind fields. `_` is the wildcard pattern.

Maybe and Outcome

```
match maybe_name
  Some(name) -> say name
  None -> say "missing"

match result
  Ok(value) -> say value
  Fail(problem) -> say problem
```

Shapes

`shape` declarations parse and format. Full interface/trait semantics are not implemented yet.

Common Mistakes

- Omitting a case in `match`.
- Using a case name that does not belong to the choice.
- Giving a pattern the wrong number of fields.

Collections and Outcomes

Lists

```
let names = ["Ada", "Grace", "Alan"]
say names[0]
say names.length()
say names.first()
```

Lists support `length`, `first`, `last`, `sort`, `reverse`, `push`, and `contains`.

Maps

```
let ages = {"Ada": 12, "Lin": 11}
say ages["Ada"]
say ages.keys()
say ages.has("Lin")
```

Maps preserve insertion order in the VM.

Maybe

Maybe of `T` is either `Some(T)` or `None`.

```
match [1, 2].first()
  Some(value) -> say value
  None -> say "empty"
```

Outcome

Outcome of `T` is either `Ok(T)` or `Fail(Text-like problem)`.

```
fn parse_grade(score: Int) -> Outcome of Text
  if score < 0
    return Fail("scores start at zero")
  return Ok("pass")
```

try

```
let fallback = try parse_grade(-1) else "?"
```

Bare `try` inside an `Outcome`-returning function propagates failure. A bare `try` on `Maybe` propagates `Fail("the value was missing")`.

Modules and Packages

Imports

```
use math
use helpers
use shop.cart
```

Stdlib imports always win. User imports load files:

- `use helpers` maps to `src/helpers.sp` in a project or sibling `helpers.sp` for scripts.
- `use shop.cart` maps to `src/shop/cart.sp`.
- `use package_name` can load `web_modules/package_name/src/lib.sp`.

Visibility

Functions cross file boundaries only when declared `public`.

```
public fn greet(name: Text)
  say "Hello, {name}!"
```

Private functions remain local to their module.

Side Effects

Only the entry file runs top-level code. Imported modules are side-effect-free. If a module contains top-level executable statements, the project checker reports E0246.

Cycles

Import cycles are errors and include the cycle path.

Limitations

- Types are project-global.
- Variant names share one global namespace.
- Transitive package dependencies are not resolved.

Testing

Spider has built-in test blocks.

```
test "adding works"
  let result = 1 + 1
  expect(result, 2)
```

Run tests with:

```
spider test .
```

Each test runs on a fresh VM. Tests cannot depend on state from another test. `expect(actual, expected)` stops the test with `EO311` if values differ.

`test` blocks do not run under `spider run`.

Best Practices

- Keep tests beside the code they describe.
- Use deterministic inputs.
- Use `random.seed` when testing random behavior.

Concurrency Surface

The parser accepts:

```
do together
  let weather = fetch_weather("Tokyo")
  let news = fetch_headlines()

spawn worker(jobs)
```

Current behavior is deliberately limited. M3 records that concurrency constructs execute sequentially until M7, and the checker warns honestly with W0003.

Do not write production Spider code that relies on parallel execution yet.

PART 5

Standard Library

Standard Library Reference

The standard library registry is `spider_hir::stdlib`. It is the single source of truth for signatures, docs, checker behavior, VM dispatch, and doc coverage.

Capabilities

Known capability names:

Capability	Meaning	Implemented module today
<code>fs</code>	Filesystem access.	<code>files</code>
<code>net</code>	Network access.	reserved
<code>env</code>	Environment access.	reserved
<code>exec</code>	Process execution.	reserved
<code>ai</code>	AI provider access.	reserved

Modules

math

Member	Signature	Description
<code>math.sqrt</code>	<code>Float -> Float</code>	Square root.
<code>math.pow</code>	<code>Float, Float -> Float</code>	Power.
<code>math.abs</code>	<code>T -> T</code>	Distance from zero.
<code>math.min</code>	<code>T, T -> T</code>	Smaller comparable value.
<code>math.max</code>	<code>T, T -> T</code>	Larger comparable value.
<code>math.floor</code>	<code>Float -> Int</code>	Round down.
<code>math.round</code>	<code>Float -> Int</code>	Round to nearest whole number.
<code>math.pi</code>	<code>Float</code>	Circle constant.

random

Member	Signature	Description
<code>random.seed</code>	<code>Int -> Nothing</code>	Set deterministic sequence seed.
<code>random.int</code>	<code>Int, Int -> Int</code>	Random integer between inclusive bounds.
<code>random.float</code>	<code>() -> Float</code>	Random float in <code>[0, 1)</code> .

files

Requires `fs`.

Member	Signature	Description
<code>files.read_text</code>	<code>Text -> Outcome of Text</code>	Read a file.
<code>files.write_text</code>	<code>Text, Text -> Outcome of Bool</code>	Write a file.
<code>files.exists</code>	<code>Text -> Bool</code>	Test path existence.
<code>files.list</code>	<code>Text -> Outcome of List of Text</code>	List directory names, sorted.

Methods

Int

- `to_float()` -> Float
- `abs()` -> Int

Float

- `to_int()` -> Int
- `round()` -> Int
- `floor()` -> Int
- `abs()` -> Float

Text

- `length()` -> Int
- `upper()` -> Text
- `lower()` -> Text
- `trim()` -> Text
- `contains(Text)` -> Bool
- `split(Text)` -> List of Text

List

- `length()` -> Int
- `first()` -> Maybe of T
- `last()` -> Maybe of T
- `sort()` -> List of T
- `reverse()` -> List of T
- `push(T)` -> Nothing
- `contains(T)` -> Bool

Map

- `length()` -> Int
- `keys()` -> List of K

- `values()` -> List of V
- `has(K)` -> Bool

Builtins

`expect(actual, expected)` is available everywhere. It is intended for `spider test`.

Diagnostics Reference

Spider diagnostics have stable codes. The implementation includes authored explanations in `spider_syntax::diagnostics`.

Syntax Diagnostics

Examples include:

- E0001: tabs in indentation.
- E0002: unexpected character.
- E0003: unterminated text literal.
- E0004: indentation does not line up.
- E0110: expected a value.
- E0115: expected a specific syntax kind.
- E0120: expected expression to end.
- E0150: nesting too deep.

Semantic Diagnostics

Examples include:

- unknown names and did-you-mean suggestions;
- duplicate names;
- assigning through immutable bindings;
- type mismatches;
- non-`Bool` conditions;
- non-exhaustive `match`;
- wrong pattern arity;
- invalid `try`;
- missing capability E0244;
- imported module top-level side effect E0246.

Runtime Panics

Code	Meaning
E0301	Integer divide by zero.
E0302	Integer overflow.
E0303	List position out of range.
E0304	Missing map key.
E0305	Sorting values with no ordering.

E0306	Unknown module member at runtime defense layer.
E0307	Recursion limit exceeded.
E0310	Runtime capability denied.
E0311	<code>expect</code> failed.

Using explain

```
spider explain E0303
```

Best Practices

- Treat diagnostics as part of the language.
- Prefer one root error over cascades.
- Use suggestions only when they point to valid code.

Language Reference

This chapter collects the implemented user-facing grammar in reference form.

Top-Level Forms

```
use path
record Name
choice Name
shape Name
fn name(params) -> Type
public fn name(params) -> Type
test "name"
statement
```

Statements

```
let name [: Type] = expr
var name [: Type] = expr
target = expr
target += expr
target -= expr
target *= expr
target /= expr
say expr
return [expr]
if expr block [else ...]
for name in expr block
while expr block
repeat expr times block
match expr arms
do together block
spawn expr
expr
```

`break` and `continue` are not implemented.

Types

```
Int
Float
Bool
Text
Nothing
List of T
Map of K to V
Maybe of T
Outcome of T
RecordName
ChoiceName
```

Expressions

Expressions include literals, names, calls, method calls, field access, indexing, lists, maps, ranges, unary operators, binary operators, `ask`, `try`, and `match` as value expression where arm results agree.

Operator Precedence

From tightest to loosest:

1. call, method call, indexing, field access
2. unary `-`
3. `*`, `/`, `%`
4. `+`, `-`
5. `to`
6. comparisons
7. `not`
8. `and`
9. `or`

CLI Reference

spider

```
spider run [--allow cap[,cap]] <file.sp|project-dir>
spider test [--allow cap[,cap]] [path]
spider new <name>
spider repl [--allow cap[,cap]]
spider fmt [--check] <paths...>
spider check <file.sp>
spider tree <file.sp>
spider tokens <file.sp>
spider explain <CODE>
spider learn <file.sp>
spider lsp
spider --version
```

Exit code `0` means success. Exit code `1` means a user program or project problem. Exit code `2` means command usage error.

web

```
web install <name>
web publish
web audit
web remove <name>
web --version
```

`web` must be run inside a project containing `web.toml`, except for help and version output.

Bytecode Reference

Silk bytecode is register based. Each function prototype has a fixed register count, bytecode instructions, and a constant table.

Program

- `protos` : function prototypes.
- `entry` : script entry prototype.
- `main` : optional zero-arity `main` .
- `n_globals` : global slot count.
- `tests` : named test prototypes.

Instructions

Instruction	Operands	Purpose
<code>LoadConst</code>	dst, const	Load constant.
<code>LoadUnit</code>	dst	Load <code>nothing</code> .
<code>Move</code>	dst, src	Copy value.
<code>LoadGlobal</code>	dst, slot	Read global.
<code>StoreGlobal</code>	slot, src	Write global.
<code>Bin</code>	op, dst, a, b	Binary operation.
<code>Neg</code>	dst, src	Numeric negation.
<code>Not</code>	dst, src	Boolean negation.
<code>Jump</code>	target	Unconditional jump.
<code>JumpIfFalse</code>	cond, target	Conditional branch.
<code>JumpIfTrue</code>	cond, target	Conditional branch.
<code>Call</code>	dst, proto, args	Call known function.
<code>CallValue</code>	dst, callee, args	Call function value.
<code>CallMethod</code>	dst, recv, name, args	Call method.
<code>CallModule</code>	dst, module, name, args	Call module function.
<code>ModuleConst</code>	dst, module, name	Load module constant.
<code>MakeList</code>	dst, items	Construct list.
<code>MakeMap</code>	dst, pairs	Construct map.
<code>MakeRange</code>	dst, lo, hi	Construct range.

<code>Index</code>	dst, base, index	Read index.
<code>IndexSet</code>	base, index, value	Write index.
<code>GetField</code>	dst, base, field	Read record field.
<code>SetField</code>	base, field, value	Write record field.
<code>MakeRecord</code>	dst, shape, fields	Construct record.
<code>MakeVariant</code>	dst, tag, fields	Construct choice value.
<code>TestTag</code>	dst, value, tag	Test variant tag.
<code>GetVariantField</code>	dst, value, pos	Read variant payload.
<code>TryUnwrap</code>	dst, value, fail_target	Unwrap <code>Ok</code> / <code>Some</code> or branch.
<code>NoneToFail</code>	reg	Convert <code>None</code> propagation to <code>Fail</code> .
<code>IterNew</code>	dst, iterable	Create iterator.
<code>IterNext</code>	item, iter, done_target	Advance iterator.
<code>Concat</code>	dst, parts	String interpolation concatenation.
<code>Say</code>	src	Print value.
<code>Ask</code>	dst, prompt	Prompt and read text.
<code>Ret</code>	src	Return value.
<code>RetUnit</code>	none	Return <code>nothing</code> .

Performance Notes

Instruction operands currently use ordinary Rust enum variants and vectors. Packing and optimization are intentionally deferred to M8.

PART 6

Internals

Compiler Architecture

Spider's implemented pipeline is:

```

flowchart LR
    Source[".sp source"] --> Lexer["Lexer"]
    Lexer --> CST["Lossless CST"]
    CST --> AST["Typed AST views"]
    AST --> Check["Name and type checker"]
    Check --> Project["Project merge and module visibility"]
    Project --> Bytecode["Silk bytecode compiler"]
    Bytecode --> VM["Silk VM"]
  
```

Lexer

The lexer produces tokens and diagnostics. It also inserts zero-width `Indent` / `Dedent` tokens. Newlines inside brackets are trivia.

Parser

The parser is recursive descent and error tolerant.

```

flowchart TD
    Tokens --> SourceFile
    SourceFile --> Statements
    Statements --> Declarations
    Statements --> Expressions
    Expressions --> Recovery["line-based recovery"]
  
```

Guarantees:

- never panic on any input;
- preserve every byte of input in the tree;
- report independent errors in source order;
- cap nesting depth.

Semantic Checker

The checker resolves names, checks mutability, infers types, checks module members against the stdlib registry, verifies match exhaustiveness, validates `try`, and applies capability policy.

```

flowchart LR
    Parse --> Names["collect names"]
    Names --> Signatures["collect signatures"]
    Signatures --> Bodies["check bodies"]
    Bodies --> Diagnostics
  
```

Project Checker

M5 adds `check_project`, which merges type names, per-module signatures, exports and constructors, then checks each module body with the merged world.

Bytecode Generator

The compiler lowers CST-shaped typed programs to register instructions. It assumes the checker has already accepted the program; inconsistencies are toolchain bugs, not user diagnostics.

Silk Virtual Machine

Silk is Spider's current execution engine.

Architecture

```
flowchart TD
    Program --> Proto["Function prototypes"]
    Proto --> Frame["Call frames"]
    Frame --> Registers
    Registers --> Instr["Instruction dispatch loop"]
    Instr --> Values
    Values --> IO["Console/files"]
```

Values

Runtime values include `Unit`, `Int`, `Float`, `Bool`, `Text`, `List`, `Map`, `Range`, `Record`, `Variant`, `FnRef`, and iterator values.

Lists, maps, records, variants, and text are reference-counted internally. Mutation uses copy-on-write so Spider assignment remains value-like.

Call Frames

Call frames are stored on an explicit heap stack. Spider recursion does not use the host call stack. The language recursion limit is 1000.

Memory

There is no exposed manual memory management. The VM uses Rust ownership and reference counting internally.

Capabilities

The VM stores an allow-set and checks capability-gated operations at the operation boundary. This is defense in depth; the checker also enforces capabilities.

Runtime Errors

Runtime panics render in Spider Explain format.

Formatter

`spider fmt` is implemented in `crates/spider-fmt`. It is canonical and has no configuration.

Rules

- Refuses to format files with parse errors.
- Normalizes whitespace and indentation.
- Preserves comments.
- Is idempotent: formatting twice produces the same output.
- Formatted output reparses cleanly.

Motivation

The formatter is part of the language contract. Users should not spend attention on style settings.

Package Loader and Registry

Loader

The loader maps imports to files and packages.

```
flowchart TD
    Use["use helpers"] --> Std{"stdlib name?"}
    Std -->|yes| Registry["stdlib registry"]
    Std -->|no| Project["src/helpers.sp"]
    Project --> Sibling["sibling helpers.sp"]
    Sibling --> Package["web_modules/helpers/src/lib.sp"]
```

Stdlib names cannot be shadowed by files. Cycles are reported with the cycle path.

Registry

The local registry copies packages into `~/.spider/registry/<name>/<version>`. Published versions are immutable. Installs copy package files into `web_modules`, write `web.lock`, update `web.toml`, and refuse capability escalation.

PART 7

Learning by Building

Tutorial

Lesson 1: Hello

```
say "Hello, world!"
```

Lesson 2: Input

```
let name = ask "What is your name?"  
say "Welcome, {name}!"
```

Lesson 3: Branches

```
let age = 12  
if age >= 13  
  say "teen"  
else  
  say "kid"
```

Lesson 4: Loops

```
var total = 0  
for i in 1 to 10  
  total += i  
say total
```

Lesson 5: Functions

```
fn double(n: Int) -> Int  
  return n * 2  
  
say double(21)
```

Lesson 6: Records and Choices

```
record Point  
  x: Float  
  y: Float  
  
choice Shape  
  Circle(radius: Float)  
  Dot
```

Lesson 7: Tests

```
test "math works"  
  expect(1 + 1, 2)
```

Challenge

Write a program that sums the even numbers from 1 to 20.

Answer:

```
var total = 0  
for n in 1 to 20  
  if n % 2 == 0  
    total += n  
say total
```

Cookbook

Print a List

```
say [1, 2, 3]
```

Sort Text

```
let names = ["Grace", "Ada", "Lin"]  
say names.sort()
```

Safely Read a File

Requires `fs`.

```
use files  
  
match files.read_text("notes.txt")  
  Ok(text) -> say text  
  Fail(problem) -> say "Could not read: {problem}"
```

Run:

```
spider run --allow fs read.sp
```

Deterministic Random

```
use random  
  
random.seed(7)  
say random.int(1, 6)
```

A Simple Parser Shape

```
fn parse_age(text: Text) -> Outcome of Int  
  if text.length() == 0  
    return Fail("empty input")  
  return Ok(21)
```

Test a Function


```
fn inc(n: Int) -> Int
  return n + 1

test "inc"
  expect(inc(1), 2)
```

Projects

These projects use implemented Spider features. Networking and real HTTP are not implemented today, so HTTP-related ideas are design exercises rather than compilable current projects.

Calculator

Build functions for add, subtract, multiply, divide, and test them.

```
fn add(a: Int, b: Int) -> Int
  return a + b

fn sub(a: Int, b: Int) -> Int
  return a - b

test "calculator"
  expect(add(2, 3), 5)
  expect(sub(5, 2), 3)
```

Todo CLI Data Model

```
record Todo
  title: Text
  done: Bool

fn label(todo: Todo) -> Text
  if todo.done
    return "[x] {todo.title}"
  return "[ ] {todo.title}"
```

JSON Parser Sketch

Current Spider can model parser results with `Outcome`, records, choices, and lists, but the stdlib has no JSON module yet.

Package Manager Exercise

Create a library with `src/lib.sp`, publish it locally with `web publish`, then install it into another project with `web install`.

Compiler Extension Exercise

Add a new stdlib method by changing the registry, checker dispatch, VM method dispatch, tests, and this book.

Exercises and Answers

This chapter collects focused exercises. They are intentionally small enough to compile against the current compiler.

Basics

1. Print your name.

```
say "Avinash"
```

2. Store a number in `let` and print double.

```
let n = 21  
say n * 2
```

3. Count from 1 to 3.

```
for i in 1 to 3  
  say i
```

4. Write a mutable counter.

```
var count = 0  
count += 1  
say count
```

5. Write a function returning text.

```
fn greet(name: Text) -> Text  
  return "Hello, {name}"
```

Control Flow

6. Classify an age.

```
let age = 12  
if age >= 13  
  say "teen"  
else  
  say "kid"
```

7. Sum a range.

```
var total = 0
for i in 1 to 10
  total += i
say total
```

8. Use `repeat`.

```
repeat 3 times
  say "again"
```

9. Use `while`.

```
var n = 3
while n > 0
  say n
  n -= 1
```

10. Match a `Maybe`.

```
match [1, 2].first()
  Some(value) -> say value
  None -> say "empty"
```

Data

11. Define a record.

```
record Point
  x: Float
  y: Float

let p = Point(1.0, 2.0)
say p.x
```

12. Define a choice.

```
choice Light
  Red
  Green

match Red
  Red -> say "stop"
  Green -> say "go"
```

13. Use a map.

```
let ages = {"Ada": 12}
say ages["Ada"]
```

14. Use a list method.

```
say [3, 1, 2].sort()
```

15. Use text methods.

```
say " Spider ".trim().upper()
```

Testing

16. Test a function.

```
fn double(n: Int) -> Int
  return n * 2

test "double"
  expect(double(4), 8)
```

17. Seed random.

```
use random
random.seed(1)
say random.int(1, 6)
```

18. Handle an outcome.

```
fn ok() -> Outcome of Int
  return Ok(1)

match ok()
  Ok(value) -> say value
  Fail(problem) -> say problem
```

19. Use fallback `try`.

```
fn fail() -> Outcome of Int
  return Fail("no")

let value = try fail() else 0
say value
```

20. Show value semantics.

```
var a = [1]
var b = a
b.push(2)
say a
say b
```

Challenge Problems

21. Write `factorial` .

```
fn factorial(n: Int) -> Int
  if n < 2
    return 1
  return n * factorial(n - 1)
```

22. Find the largest list element using `.sort().last()` .

```
match [3, 9, 1].sort().last()
  Some(value) -> say value
  None -> say "empty"
```

23. Convert a score to a grade with `Outcome` .

```
fn grade(score: Int) -> Outcome of Text
  if score < 0
    return Fail("negative")
  if score >= 90
    return Ok("A")
  return Ok("B")
```

24. Add a test for `grade` .

```
test "grade"
  expect(grade(95), Ok("A"))
```

25. Explain why `files.read_text` needs a capability.

Answer: it observes the host filesystem, so Safe Mode requires `fs` .

PART 8

Contributor Guide

Contributing

Repository Structure

Path	Purpose
<code>crates/spider-syntax</code>	Lexer, parser, CST, AST views, diagnostics.
<code>crates/spider-hir</code>	Semantic checker, types, stdlib registry, manifest caps.
<code>crates/spider-silk</code>	Bytecode compiler, VM, runtime values.
<code>crates/spider-fmt</code>	Formatter.
<code>crates/spider-web</code>	Project loader and <code>web</code> package manager.
<code>crates/spider-lsp</code>	Language server.
<code>crates/spider-cli</code>	<code>spider</code> command.
<code>corpus</code>	Golden syntax, format, check, run fixtures.
<code>docs</code>	SRS, LDD, ADRs, milestone notes.
<code>website</code>	Current static website assets.

Test Commands

```
cargo test --workspace
```

Standards

- Preserve parser losslessness.
- Add diagnostics with authored explanations.
- Update corpus goldens when syntax or formatting changes intentionally.
- Keep stdlib registry, checker, VM dispatch, tests, and docs in sync.
- Never document future features as implemented.

Review Checklist

- Does it parse broken input without panic?
- Does it preserve source text?
- Does `spider fmt` stay idempotent?
- Does the checker avoid cascaded diagnostics?
- Does the VM preserve value semantics?
- Are capabilities enforced at check time and runtime?

Adding Language Features

Adding Syntax

1. Add token kinds in `spider-syntax`.
2. Extend the lexer if new spelling is required.
3. Extend the parser with recovery behavior.
4. Add corpus examples and tree snapshots.
5. Update the formatter.
6. Add diagnostics and explanations.
7. Update this book.

Adding Type Rules

1. Extend `ty` if needed.
2. Add inference/checking rules.
3. Add positive and negative corpus examples.
4. Prevent diagnostic cascades.

Adding Bytecode

1. Add an `Instr` variant.
2. Lower from the compiler.
3. Execute it in the VM loop.
4. Add runtime tests and semantics cases.
5. Document the instruction in the bytecode reference.

Adding Standard Library APIs

1. Add the signature and doc to `spider_hir::stdlib`.
2. Add VM dispatch.
3. Add checker or method dispatch tests.
4. Add examples.
5. Update the standard library chapter.

PART

Appendix

Grammar Sketch

This is an implementation-oriented grammar sketch, not a formal complete BNF.

```

file      = item*
item      = use | record | choice | shape | fn | test | stmt
use       = "use" ident ( "." ident ) *
record    = "record" name newline indent field* dedent
choice    = "choice" name newline indent variant* dedent
shape     = "shape" name newline indent fn-signature* dedent
fn        = ["public"] "fn" name "(" params ")" ["->" type] [where] block
test      = "test" string block
stmt      = let | var | assign | say | return | if | for | while | repeat
           | match | do-together | spawn | expr
block     = newline indent stmt* dedent
type      = name | "List" "of" type | "Maybe" "of" type
           | "Outcome" "of" type | "Map" "of" type "to" type
expr      = or-expr

```

The parser source remains authoritative.

Keywords and Precedence

Hard Keywords

let, var, fn, if, else, for, in, to, while, repeat, match, try, use, return, say, ask, spawn, do, and, or, not, true, false, of, is, and public.

Soft Keywords

record, choice, shape, test, times, together, and where.

Precedence

1. postfix: calls, methods, fields, indexes
2. unary minus
3. multiplication, division, modulo
4. addition, subtraction
5. range to
6. comparisons
7. not
8. and
9. or

Glossary

Capability: A named permission such as `fs` that a program or package must declare before using gated APIs.

CST: Concrete syntax tree. Spider's CST is lossless.

HIR: Higher-level semantic representation. The repository uses `spider-hir` for checking even though the public structure is still close to syntax.

Outcome: A typed expected-failure value, either `Ok(value)` or `Fail(problem)`.

Maybe: An optional value, either `Some(value)` or `None`.

Silk: Spider's bytecode virtual machine.

Soft keyword: A word that is only a keyword in specific grammar positions.

Value semantics: Assignment behaves like a copy.

FAQ

Is Spider implemented?

Partially. The parser, checker, formatter, VM, CLI, local package manager, and LSP are implemented. Native build, public registry, real concurrency, and docs generation are future work.

Is Spider English?

No. Spider uses readable words but has a strict grammar.

Does Spider have null?

No current null value exists. Use `Maybe`.

Does Spider use exceptions?

Expected failures use `Outcome`. Runtime panics exist for bugs such as divide-by-zero or out-of-range indexing.

Can packages touch files by default?

No. File APIs require the `fs` capability.

Does `spider build` work?

No. It is a roadmap item.

Troubleshooting

`spider: build is not a command`

`spider build` is not implemented yet.

E0001 Tabs in indentation

Use spaces for indentation.

E0244 Missing capability

Add the capability to `web.toml` or pass `--allow` for a script run.

E0303 List position

List indexes start at 0. Check the list length before indexing.

Package install refused

The dependency asks for capabilities your project does not allow. Add the capability only if you trust the package.

Validation Report

Generated from repository inspection on 2026-07-06.

Build Verification

- `cargo test --workspace` : passed.
- `cargo install mdbook` : completed, installing mdBook 0.5.3.
- `mdbook build book` : passed.
- HTML output path: `book/book` .

Counts

- Chapters in this mdBook: 35.
- Spider code examples written in this book: 70+.
- Exercises and challenge prompts: 25.
- Standard-library registry entries documented: 39.
- Bytecode instructions documented: 35.

This first generated book is not yet a 400-600 page manuscript and does not yet contain 600 examples or 300 exercises. It is a complete publishable documentation skeleton with substantive content for every implemented area, ready for expansion as the language grows.

Verified Implemented CLI Commands

- `spider run`
- `spider test`
- `spider new`
- `spider repl`
- `spider lsp`
- `spider learn`
- `spider fmt`
- `spider check`
- `spider tree`
- `spider tokens`
- `spider explain`
- `web install`
- `web publish`
- `web audit`
- `web remove`

Undocumented APIs

No current stdlib registry entries are intentionally omitted from the standard library chapter.

Undocumented Compiler Features

The parser and checker include internal recovery details and many diagnostics not expanded one by one in this first pass. The diagnostics chapter names the major families and runtime codes.

Undocumented VM Instructions

No current `Instr` variants are intentionally omitted from the bytecode reference.

Known Discrepancies

- Roadmap commands `spider build`, `spider doc`, and `spider package` are not implemented.
- `break` and `continue` are requested in the original documentation brief but are not implemented.
- Sets, tuples, traits, interfaces, closures, anonymous functions, native AOT, real concurrency, HTTP server APIs, JSON APIs, regex APIs, env APIs, exec APIs, and AI APIs are not implemented in the current code.
- `shape` parses but does not yet provide full trait/interface semantics.
- Website documentation, PDF, EPUB, and DOCX outputs require additional build tooling beyond this mdBook source.



Spider — spin something.

Programming should be as easy as speaking — without ever becoming a toy. Spider pairs a surface simple enough for a beginner's first afternoon with a core sound enough for production: a teaching compiler, memory safety, capability security, and a fast virtual machine.

spider-lang.vercel.app • avixr.in

github.com/kvAvinashsarma-dev/Spider-Programming-Language

KV AVINASH SARMA · EARLY EDITION · V0.1.0